

科沃斯机器人股份有限公司Ecovacs



科沃斯机器人股份有限公司Ecovacs

科沃斯机器人股份有限公司Ecovacs

科沃斯机器人股份有限公司Ecovacs

科沃斯机器人股份有限公司Ecovacs

科沃斯机器人股份有限公司Ecovacs

Linux TWI 开发指南

科沃斯机器人股份有限公司Ecovacs

科沃斯机器人股份有限公司Ecovacs

科沃斯机器人股份有限公司Ecovacs

科沃斯机器人股份有限公司Ecovacs

科沃斯机器人股份有限公司Ecovacs

科沃斯机器人股份有限公司Ecovacs

科沃斯机器人股份有限公司Ecovacs

科沃斯机器人股份有限公司Ecovacs

版本号: 1.3
发布日期: 2023.11.17

科沃斯机器人股份有限公司Ecovacs

科沃斯机器人股份有限公司Ecovacs

版本历史

版本号	日期	制/修订人	内容描述
1.0	2019.12.31	AWA1637	建立初版。
1.1	2021.04.10	XAA0193	1. 适用列表中增加了新的 IC。2. 对 5.4 下的 dts 进行了更新。
1.2	2021.04.29	XAA0193	根据评审意见再次进行修改。
1.3	2023.11.17	XAA0309	根据最新环境全文修改。

目 录

1 前言	1
1.1 文档简介	1
1.2 目标读者	1
1.3 适用范围	1
2 模块介绍	2
2.1 模块功能介绍	2
2.2 相关术语介绍	2
2.2.1 硬件术语	2
2.2.2 软件术语	2
2.3 模块配置介绍	3
2.3.1 device tree 默认配置	3
2.3.2 board.dts 板级配置	4
2.3.3 kernel menuconfig 配置	5
2.4 源码模块结构	8
2.5 驱动框架介绍	9
3 模块配置	11
3.1 master 模式	11
3.1.1 engine 模式配置	11
3.1.2 drv 模式配置	11
3.1.3 传输速率配置	11
3.2 slave 模式	12
3.2.1 menuconfig 配置	12
3.2.2 dts 配置	12
4 模块接口说明	13
4.1 i2c-core 接口	13
4.1.1 i2c_transfer()	13
4.2 i2c 用户态调用接口	13
4.2.1 i2cdev_open()	13
4.2.2 i2cdev_read()	14
4.2.3 i2cdev_write()	14
4.2.4 i2cdev_ioctl()	14
5 模块使用范例	16
5.1 利用用户态接口读写 TWI 设备	16
6 调试方法	17
6.1 i2c-tools 调试工具	17

6.1.1	i2cdetect	17
6.1.1.1	参数说明	17
6.1.1.2	使用示例	17
6.1.2	i2cdump	18
6.1.2.1	参数说明	18
6.1.2.2	使用示例	18
6.1.3	i2cset	18
6.1.3.1	参数说明	18
6.1.3.2	使用示例	19
6.1.4	i2cget	19
6.1.4.1	参数说明	19
6.1.4.2	使用示例	20
6.1.5	i2ctransfer	20
6.1.5.1	参数说明	20
6.1.5.2	使用示例	20
6.2	调试节点	21
6.2.1	info	21
6.2.2	status	21
6.2.3	freq	21
6.2.4	twi_mode	21
6.2.5	debug_chan	21
7	FAQ	22
7.1	TWI 数据未完全发送	22
7.2	TWI 起始信号无法发送	22
7.3	TWI 终止信号无法发送	23
7.4	TWI 传送超时	23

插图

图 2-1	Device Driver	6
图 2-2	I2C support	6
图 2-3	I2C device interface	7
图 2-4	2C HardWare Bus support	7
图 2-5	SUNXI I2C controller	8
图 2-6	TWI 模块结构框图	9

1 前言

1.1 文档简介

介绍 Sunxi 平台上 TWI 驱动接口与调试方法，为 TWI 模块开发提供参考。

1.2 目标读者

TWI 模块内核层以及应用层的开发、维护人员。

1.3 适用范围

表 1-1: 适用产品列表

内核版本	驱动文件
Linux-4.9	i2c-sunxi.c
Linux-5.4	i2c-sunxi.c

2 模块介绍

2.1 模块功能介绍

全志公司的 twi 总线兼容 i2c 总线协议，是一种简单、双向二线制同步串行总线。它只需要两根线即可在连接于总线上的器件之间传送信息。TWI 控制器支持的标准通信速率为 100kbps，最高通信速率可以达到 400kbps。全志的 twi 控制器支持一下功能：

- 支持主机模式和从机模式；
- 主机模式下支持 dma 传输；
- 主机模式下在多个主机的模式下支持总线仲裁；
- 主机模式下支持时钟同步，位和字节等待；
- 从机模式下支持地址检测中断；
- 支持 7bit 从机地址和 10bit 从机地址；
- 支持常规的 i2c 协议模式和自定义传输模式；

sunxi 平台支持多路 TWI，包含 TWI 与 S_TWI。

2.2 相关术语介绍

2.2.1 硬件术语

表 2-1: 硬件术语

相关术语	解释说明
TWI	Two Wire Interface，全志平台兼容 I2C 标准协议的总线控制器。

2.2.2 软件术语

表 2-2: 软件术语

相关术语	解释说明
Sunxi	全志科技使用的 linux 开发平台。
I2C_dapter	linux 内核中 I2C 总线适配器的抽象定义；又称为 IIC 总线的控制器。
I2C_algorithm	linux 内核中 I2C 总线通信的抽象定义。描述 I2C 总线适配器与 I2C 设备之间的通信方法。
I2C Client	linux 内核中 I2C 设备的抽象定义。
I2C Driver	linux 内核中 I2C 设备驱动的抽象定义。

2.3 模块配置介绍

在不同的 Sunxi 硬件平台中，TWI 控制器的数目不同；但对于同一块板子上的每一个 TWI 控制器来说，模块配置类似，本小节展示 Sunxi 平台上的 TWI0 控制器配置（其他 TWI 控制器配置类似）。

2.3.1 device tree 默认配置

设备树中存在的是该类芯片所有平台的模块配置，设备树文件的路径为：{linux-ver}/arch/arm64（32 位平台为 arm）/boot/dts/sunxi(32 位系统无这目录)/xxxx.dtsi(CHIP 为研发代号，如 sun50iw10p1 等)。

在 linux-4.9 中，TWI 总线的设备树配置如下所示：

```
1 twi0: twi@0x05002000{
2     #address-cells = <1>;
3     #size-cells = <0>;
4     compatible = "allwinner,sun50i-twi"; // 具体的设备，用于驱动和设备的绑定
5     device_type = "twi0"; // 设备节点名称，用于sys_config.fex匹配
6     reg = <0x0 0x05002000 0x0 0x400>; // TWI0总线寄存器配置
7     interrupts = <GIC_SPI 6 IRQ_TYPE_LEVEL_HIGH>; // TWI0总线中断号、中断类型
8     clocks = <&clk_twi0>; // 设备使用的时钟
9     clock-frequency = <400000>; // TWI0控制器的时钟频率
10    pinctrl-names = "default", "sleep"; // TWI0控制器使用的Pin脚名称，其中default为正常通信时的引脚配置，
    sleep为睡眠时的引脚配置
11    pinctrl-0 = <&twi0_pins_a>; // TWI0控制器default时使用的pin脚配置
12    pinctrl-1 = <&twi0_pins_b>; // TWI0控制器sleep时使用的pin脚配置
13    twi_drv_used = <1>; // 使用DMA传输数据
14    status = "disabled"; // TWI0控制器是否使能
15 }
```

clk_twi0 为时钟的配置路径为 arch/arm64（32 位平台为 arm）/boot/dts/sunxi/XXXX-clk.dtsi(CHIP 为研发代号，如 sun50iw10p1 等)，具体配置如下所示：

```
1 clk_twi0: twi0{
2     #clock-cells = <0>;
3     compatible = "allwinner,periph-clock";
4     clock-output-names = "twi0"; //指定clock名称，用于匹配clock配置
5 }
```

在 linux-5.4 中，TWI 的配置与 linux-4.9 内核配置有些不同，区别主要体现在 clock 和 dma 的配置上：

```
1 twi0: twi@5002000 {
2     #address-cells = <1>;
3     #size-cells = <0>;
4     compatible = "allwinner,sun50i-twi"; /* 具体的设备，用于驱动和设备的绑定 */
5     device_type = "twi0"; /* 设备节点名称，用于sys_config.fex匹配 */
6     reg = <0x0 0x02502000 0x0 0x3ff>; /* 设备使用的地址 */
7     interrupts = <GIC_SPI 10 IRQ_TYPE_LEVEL_HIGH>; /* 设备使用的中断：中断号、中断类型 */
8     clocks = <&ccu CLK_TWI0>; /* 设备使用的时钟 */
9     clock-names = "bus"; /* 使用的时钟名 */
10    resets = <&ccu RST_BUS_TWI0>; /* 设备使用的复位时钟 */
11    dmas = <&dma 43>, <&dma 43>; /* 设备使用的dma通道号 */
12    dma-names = "tx", "rx"; /* 设备使用的dma通道名 */
13    status = "disabled"; /* 设备是否使用，dtsi中设为disabled，在board.dts中设置 */
14};
```

为了在 TWI 总线驱动代码中区分每一个 TWI 控制器，需要在 Device Tree 中的 aliases 节点中为每一个 TWI 节点指定别名：

```
1 aliases {
2     soc_twi0 = &twi0;
3     soc_twi1 = &twi1;
4     soc_twi2 = &twi2;
5     soc_twi3 = &twi3;
6     ...
7};
```

别名形式为字符串 “twi” 加连续编号的数字，在 TWI 总线驱动程序中可以通过 of_alias_get_id() 函数获取对应 TWI 控制器的数字编号，从而区别每一个 TWI 控制器。

2.3.2 board.dts 板级配置

board.dts 用于保存每一个板级平台的设备信息（如 demo 板，perf1 板，ver1 板等等），里面的配置信息会覆盖上面的 device tree 默认配置信息。board.dts 的路径为 longan/device/config/chips/IC/configs/BOARD/board.dts，

在 linux-4.9 中，对应 board.dts 里面 TWI0 的具体配置如下：

```
1 twi0_pins_a: twi0@0 {
2     allwinner,pins = "PA0", "PA1";
3     allwinner,pname = "twi0_scl", "twi0_sda";
4     allwinner,function = "twi0";
5     allwinner,muxsel = <4>;
6     allwinner,drive = <1>;
7     allwinner,pull = <0>;
8 };
9
10 twi0_pins_b: twi0@1 {
11     allwinner,pins = "PA0", "PA1";
12     allwinner,function = "io_disabled";
13     allwinner,muxsel = <7>;
```

```

14 allwinner,drive = <1>;
15 allwinner,pull = <0>;
16 };
17
18 twi0: twi@0x05002000{
19     clock-frequency = <400000>; //i2c时钟频率为400K
20     pinctrl-0 = <&twi0_pins_a>;
21     pinctrl-1 = <&twi0_pins_b>;
22     status = "okay";           //使能TWI0
23 };

```

在 linux-5.4 中，对应 board.dts 里面 TWI0 的具体配置如下：

```

1 &twi0{
2     clock-frequency = <400000>; /* TWI控制器的时钟频率 */
3     pinctrl-names = "default", "sleep"; /* 设备使用的pin脚名称 */
4     pinctrl-0 = <&twi0_pins_default>; /* 设备使用的pin脚配置 (default) */
5     pinctrl-1 = <&twi0_pins_sleep>; /* 设备使用的pin脚配置 (sleep) */
6     twi_drv_used = <1>; /* 是否启用drv模式传输数据：0-engine模式，1-drv模式 */
7     twi-supply = <&reg_dcdc1>; /* regulator相关配置 */
8     status = "okay"; /* 设备是否使用 */
9     eeprom@50 { /* eeprom从设备 */
10         compatible = "atmel,24c16"; /* eeprom从设备对应的设备驱动compatible，用于驱动和设备的绑定 */
11         reg = <0x50>; /* eeprom从设备地址为0x50 */
12         status = "disabled"; /* eeprom从设备status */
13     };

```

其中，TWI 速率由 “clock-frequency” 属性配置，最大支持 400K。

对于 TWI 设备，可以把设备节点填充作为 Device Tree 中相应 TWI 控制器的子节点。TWI 控制器驱动的 probe 函数透过 of_i2c_register_devices()，自动展开作为其子节点的 TWI 设备。

对于 twi0 中引用的 pin 口，具体的配置如下：

```

twi0_pins_a: twi0@0 {
    pins = "PB0", "PB1"; /* 使用的引脚 */
    function = "twi0"; /* 引脚功能 */
    drive-strength = <10>; /* 引脚配置 */
    bias-pull-up;
};

twi0_pins_b: twi0@1 {
    pins = "PB0", "PB1"; /* 使用的引脚 */
    function = "gpio_in"; /* 引脚功能 */
};

```

2.3.3 kernel menuconfig 配置

在 longan 中，linux-4.9 在命令行进入内核根目录 (/kernel/linux-4.9)，执行 make ARCH=arm64 menuconfig (32 位平台执行：make ARCH=arm menuconfig) 进入配置主界面，并按以下步骤操作 (linux-5.4 在根目录中执行 ./build.sh menuconfig)

在 tina 中，可以直接在根目录里面执行 make kernel_menuconfig 进入 menuconfig 配置界面（以下 menuconfig 配置流程在 linux-4.9 与 linux-5.4 一致）。

- 1. 选择 Device Drivers 选项进入下一级配置，如下图所示。

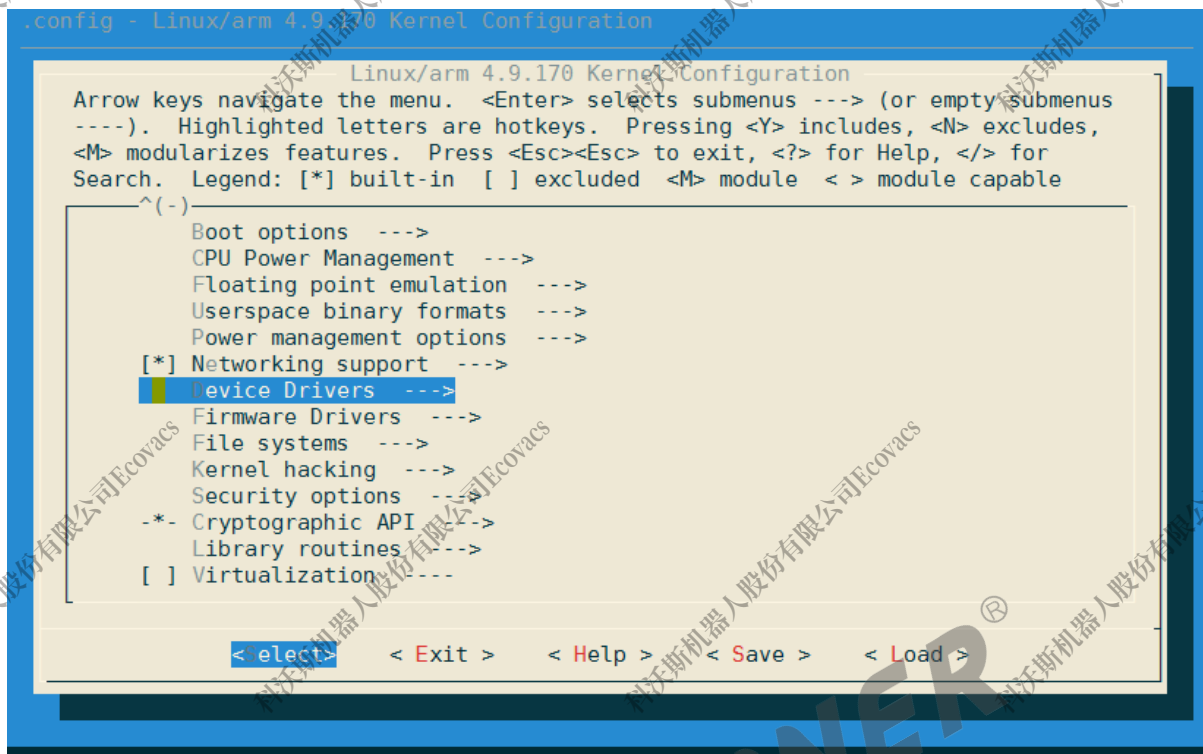


图 2-1: Device Driver

- 2. 选择 I2C support 选项，进入下一级配置，如下图所示。

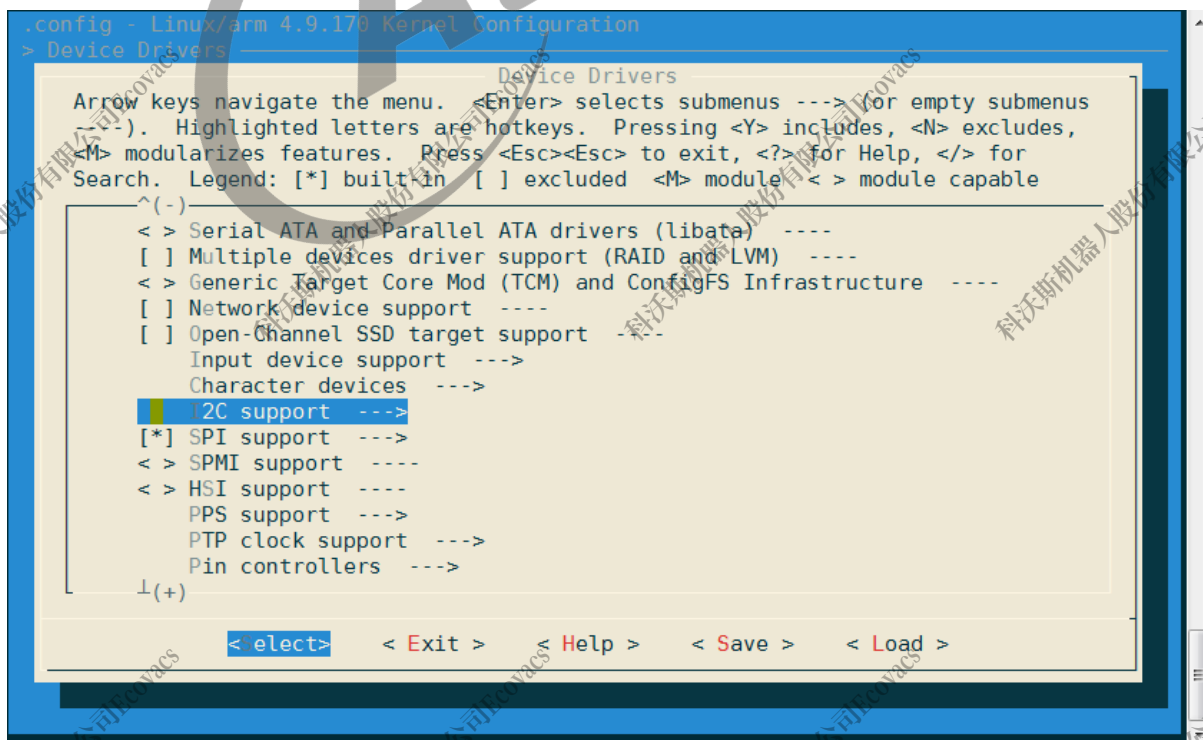


图 2-2: I2C support

- 3. 配置用户 I2C 接口，选择 I2C device interface，如下图所示。

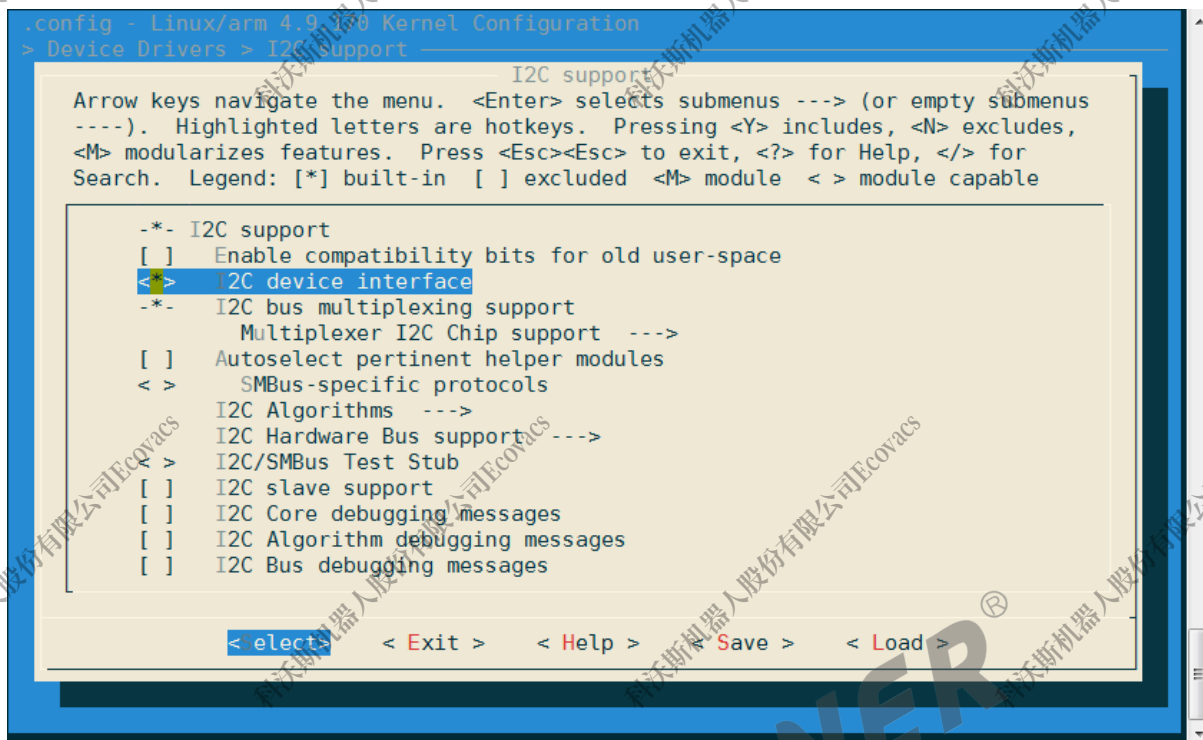


图 2-3: I2C device interface

- 4. 选择 I2C HardWare Bus support 选项，进入下一级配置，如下图所示。

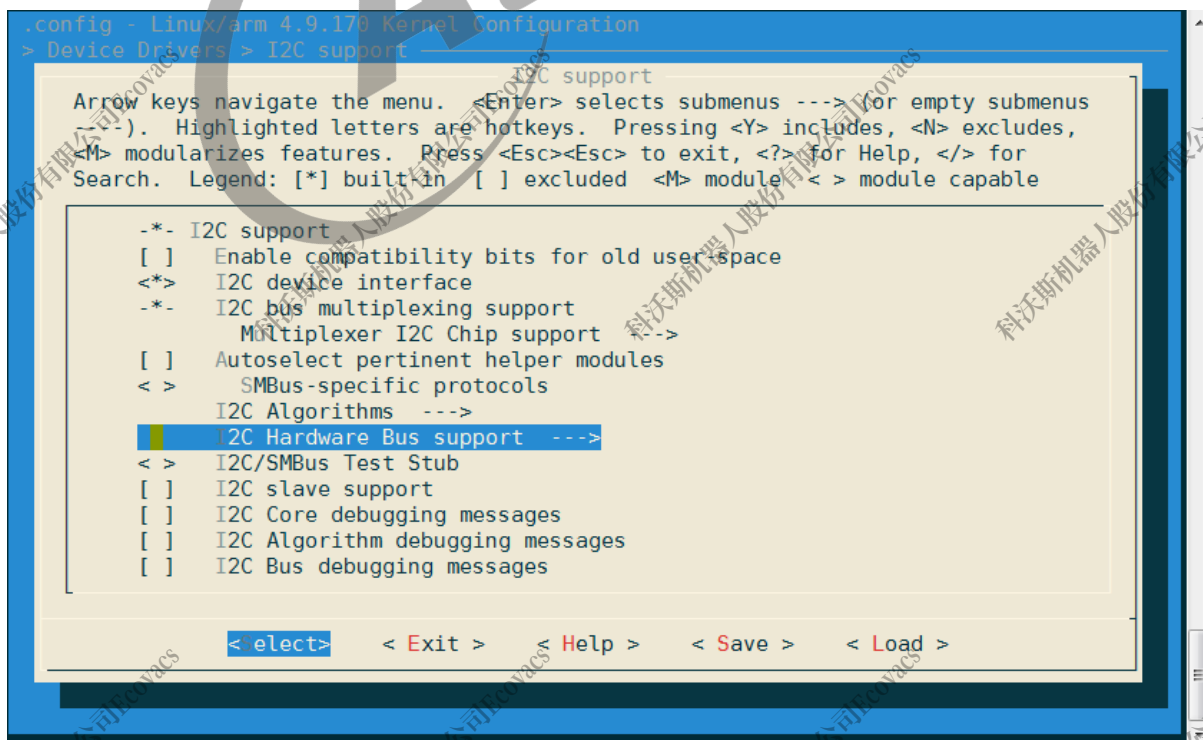


图 2-4: I2C HardWare Bus support

- 5. 选择 SUNXI I2C controller 选项，可选择直接编译进内核，也可编译成模块。如下图所示。

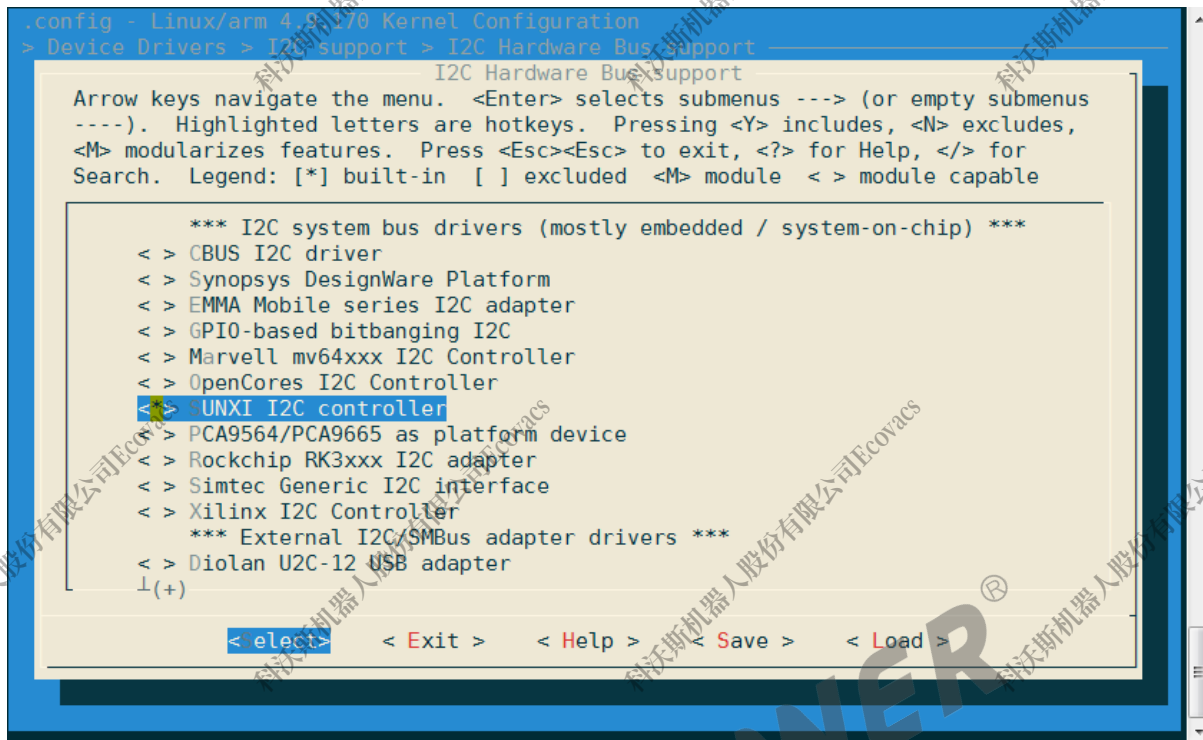


图 2-5: SUNXI I2C controller

2.4 源码模块结构

在 linux-4.9 环境中，I2C 总线驱动的源代码位于内核在 drivers/i2c/busses 目录下：

```
kernel/linux-4.9/drivers/i2c/
├── busses
│   ├── i2c-sunxi.c    // Sunxi平台的I2C控制器驱动代码
│   └── i2c-sunxi.h    // 为Sunxi平台的I2C控制器驱动定义了一些宏、数据结构
├── i2c-core.c        // I2C子系统核心文件,提供相关的接口函数
└── i2c-dev.c         // I2C子系统的设备相关文件,用以注册相关的设备文件,方便调试
```

在 linux-5.4 环境中，I2C 总线驱动的源代码位于内核在 drivers/i2c/busses 目录下：

```
kernel/linux-5.4/drivers/i2c/
├── busses
│   ├── i2c-sunxi.c    // Sunxi平台的I2C控制器驱动代码
│   └── i2c-sunxi.h    // 为Sunxi平台的I2C控制器驱动定义了一些宏、数据结构
├── i2c-core.c        // I2C子系统核心文件,提供相关的接口函数
└── i2c-dev.c         // I2C子系统的设备相关文件,用以注册相关的设备文件,方便调试
```

2.5 驱动框架介绍

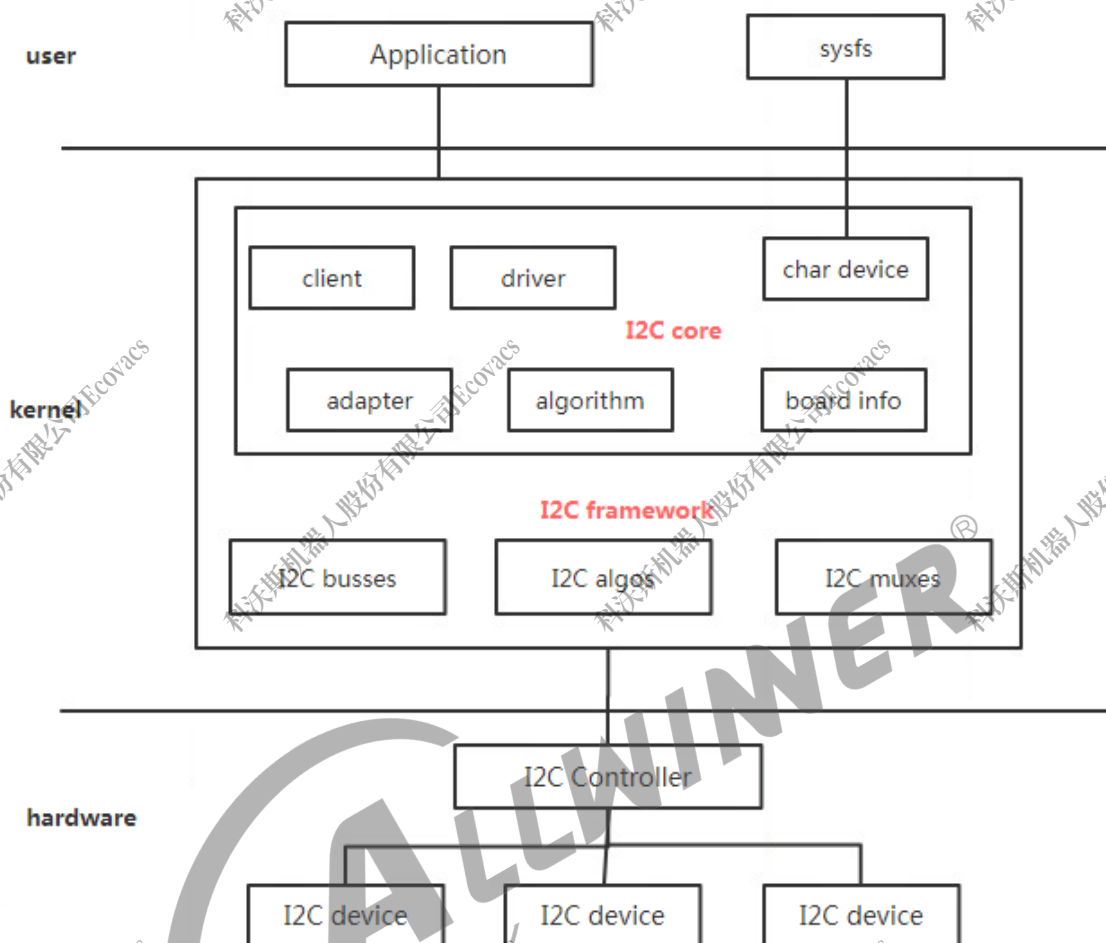


图 2-6: TWI 模块结构框图

Linux 中 I2C 体系结构上图所示，图中用分割线分成了三个层次：

1. 用户空间，包括所有使用 I2C 设备的应用程序；
2. 内核，也就是驱动部分；
3. 硬件，指实际物理设备，包括了 I2C 控制器和 I2C 外设。

其中，Linux 内核中的 I2C 驱动程序从逻辑上又可以分为 6 个部分：

1. I2C framework 提供一种“访问 I2C slave devices”的方法。由于这些 slave devices 由 I2C controller 控制，因而主要由 I2C controller 驱动实现这一目标。

2. 经过 I2C framework 的抽象，用户可以不用关心 I2C 总线的技术细节，只需要调用系统的接口，就可以与外部设备进行通信。正常情况下，外部设备是位于内核态的其它 driver（如触摸屏，摄像头等等）。I2C framework 也通过字符设备向用户空间提供类似的接口，用户空间程序可以通过该接口访问从设备信息。
3. 在 I2C framework 内部，有 I2C core、I2C busses、I2C algos 和 I2C muxes 四个模块。
4. I2C core 使用 I2C adapter 和 I2C algorithm 两个子模块抽象 I2C controller 的功能，使用 I2C client 和 I2C driver 抽象 I2C slave device 的功能（对应设备模型中的 device 和 device driver）。另外，基于 I2C 协议，通过 smbus 模块实现 SMBus（System Management Bus，系统管理总线）的功能。
5. I2C busses 是各个 I2C controller drivers 的集合，位于 drivers/i2c/busses/目录下，i2c-sunxi-test.c、i2c-sunxi.c、i2c-sunxi.h。
6. I2C algos 包含了一些通用的 I2C algorithm，所谓的 algorithm，是指 I2C 协议的通信方法，用于实现 I2C 的 read/write 指令，一般情况下，都是由硬件实现，不需要特别关注该目录。

3 模块配置

3.1 master 模式

TWI 控制器默认工作在 master 模式，无需其他额外配置。TWI 支持 engine 和 drv 两种工作模式，drv 模式是基于 engine 模式的优化，将原本一笔完整传输所需要的 5 次中断合并为 1 笔中断完成，提高传输效率，建议使用 drv 模式。

3.1.1 engine 模式配置

工作在 engine 模式，主要是配置 `twi_drv_used = <0>` 属性，TWI 模块在 **dts 文件** 中的具体配置如下：

```
&twi0 {
    clock-frequency = <400000>; /* TWI控制器的时钟频率 */
    pinctrl-names = "default", "sleep"; /* 设备使用的pin脚名称 */
    pinctrl-0 = <&twi0_pins_default>; /* 设备使用的pin脚配置 (default) */
    pinctrl-1 = <&twi0_pins_sleep>; /* 设备使用的pin脚配置 (sleep) */
    twi_drv_used = <0>; /* 是否启用drv模式传输数据：0-engine模式 */
    no_suspend = <1>; /* 默认为0，无需配置；当pmu依赖twi时需要置1，标识twi在休眠唤醒阶段不休眠 */
    status = "okay"; /* 设备是否使用 */
};
```

3.1.2 drv 模式配置

工作在 drv 模式，主要是配置 `twi_drv_used = <1>` 属性，TWI 模块在 **dts 文件** 中的具体配置如下：

```
&twi0 {
    clock-frequency = <400000>; /* TWI控制器的时钟频率 */
    pinctrl-names = "default", "sleep"; /* 设备使用的pin脚名称 */
    pinctrl-0 = <&twi0_pins_default>; /* 设备使用的pin脚配置 (default) */
    pinctrl-1 = <&twi0_pins_sleep>; /* 设备使用的pin脚配置 (sleep) */
    twi_drv_used = <1>; /* 是否启用drv模式传输数据：1-drv模式 */
    no_suspend = <1>; /* 默认为0，无需配置；当pmu依赖twi时需要置1，标识twi在休眠唤醒阶段不休眠 */
    status = "okay"; /* 设备是否使用 */
};
```

3.1.3 传输速率配置

工作在标准模式，时钟频率为 100k，配置 `clock-frequency = <100000>;`；

工作在快速模式，时钟频率为 400k，主要配置 clock-frequency = <400000>。

TWI 模块在 **dts 文件** 中的具体配置如下：

```
&twi0 {  
    clock-frequency = <400000>; /* TWI控制器的时钟频率，默认为400k */  
    ...  
};
```

3.2 slave 模式

TWI 控制器支持作为从机使用，但是默认工作在 master 模式，slave 模式需要特殊配置。

3.2.1 menuconfig 配置

需要打开以下两个宏配置选项。

CONFIG_TWI_SLAVE, CONFIG_TWI_SLAVE_EEPROM should be configured as y or M.

3.2.2 dts 配置

slave 模式时 TWI 控制器需要配置为 engine 模式，并且在 dts 中注册为 slave，具体配置如下：

TWI 模块在 **dts 文件** 中的具体配置如下：

```
&twi0 {  
    ...  
    twi_drv_used = <0>; /* slave must work in engine-mode */  
    status = "okay";  
    slave {  
        compatible = "slave-24c02"; /* Slave device drivers provided by the kernel */  
        reg = <0x50>; /* slave_addr, 可以修改 */  
        status = "okay";  
    };  
};
```

按上述配置完成后，该 TWI 控制器即注册为一个从设备，设备地址为上述定义的 slave_addr，该 TWI 控制器与其他 master 的 sck 和 sda 连接后可以正常通信。

4 模块接口说明

4.1 i2c-core 接口

4.1.1 i2c_transfer()

- 函数原型：int i2c_transfer(struct i2c_adapter *adap, struct i2c_msg *msgs, int num)
- 作用：完成 I2C 总线和 I2C 设备之间的一定数目的 I2C message 交互。
- 参数：
 - adap：指向所属的 I2C 总线控制器；
 - msgs：i2c_msg 类型的指针；
 - num：表示一次需要处理几个 I2C msg
- 返回：
 - >0：已经处理的 msg 个数；
 - <0：失败；

4.2 i2c 用户态调用接口

i2c 的操作在内核中是当做字符设备来操作的，可以通过利用文件读写接口（open，write，read，ioctl）等操作内核目录中的/dev/i2c-* 文件来调用相关的接口，i2c 相关的操作定义在 i2c-dev.c 里面，本节将介绍比较重要的几个接口：

4.2.1 i2cdev_open()

- 函数原型：static int i2cdev_open(struct inode *inode, struct file *file)
- 作用：程序（C 语言等）使用 open(file) 时调用的函数。打开一个 i2c 设备，可以像文件读写的方式往 i2c 设备中读写数据
- 参数：
 - inode：inode 节点；
 - file：file 结构体；
- 返回：文件描述符

4.2.2 i2cdev_read()

- 函数原型：static ssize_t i2cdev_read(struct file *file, char __user *buf, size_t count, loff_t *offset)
- 作用：程序（C 语言等）调用 read() 时调用的函数。像往文件里面读数据一样从 i2c 设备中读取数据。底层调用 i2c_xfer 传输数据
- 参数：
 - file: file 结构体；
 - buf, 写数据 buf；
 - offset, 文件偏移。
- 返回：
 - 非空：返回读取的字节数；
 - <0: 失败；

4.2.3 i2cdev_write()

- 函数原型：static ssize_t i2cdev_write(struct file *file, const char __user *buf, size_t count, loff_t *offset)
- 作用：程序（C 语言等）调用 write() 时调用的函数。像往文件里面写数据一样往 i2c 设备中写数据。底层调用 i2c_xfer 传输数据
- 参数：
 - file: file 结构体；
 - buf: 读数据 buf；
 - offset, 文件偏移。
- 返回：
 - 0: 成功；
 - <0: 失败；

4.2.4 i2cdev_ioctl()

- 函数原型：static long i2cdev_ioctl(struct file *file, unsigned int cmd, unsigned long arg)
- 作用：程序（C 语言等）调用 ioctl() 时调用的函数。像对文件管理 i/o 一样对 i2c 设备管理。该功能比较强大，可以修改 i2c 设备的地址，往 i2 设备里面读写数据，使用 smbus 等等，详细的可以查阅该函数。
- 参数：
 - file: file 结构体；
 - cmd: 指令；

— arg：其他参数。

- 返回：

- 0：成功；
- <0：失败；



5 模块使用范例

5.1 利用用户态接口读写 TWI 设备

如果配置了 i2c devices interface，可以直接利用文件读写函数来操作 I2C 设备。下面这个程序直接读取 /dev/i2c-* 来读写 i2c 设备：

```
#include <sys/ioctl.h>
#include <fcntl.h>
#include <linux/i2c-dev.h>
#include <linux/i2c.h>
#define CHIP "/dev/i2c-1"
#define CHIP_ADDR 0x50
int main()
{
    unsigned char rddata;
    unsigned char rdaddr[2] = {0, 0}; /* 将要读取的数据在芯片中的偏移量 */
    unsigned char wrbuf[3] = {0, 0, 0x3c}; /* 要写的的数据，头两字节为偏移量 */
    printf("hello, this is i2c tester\n");
    int fd = open(CHIP, O_RDWR);
    if (fd < 0)
    {
        printf("open \"CHIP\" failed\n");
        goto exit;
    }
    if (ioctl(fd, I2C_SLAVE_FORCE, CHIP_ADDR) < 0)
    {
        /* 设置芯片地址 */
        printf("ioctl:set slave address failed\n");
        goto close;
    }
    printf("input a char you want to write to E2PROM\n");
    wrbuf[2] = getchar();
    printf("write return:%d, write data:%x\n", write(fd, wrbuf, 3), wrbuf[2]);
    sleep(1);
    printf("write address return: %d\n", write(fd, rdaddr, 2)); /* 读取之前首先设置读取的偏移量 */
    printf("read data return:%d\n", read(fd, &rddata, 1));
    printf("rddata: %c\n", rddata);
    close(fd);
exit:
    return 0;
}
```

6 调试方法

6.1 i2c-tools 调试工具

i2c-tools 是一个开源工具，专门用来调试 TWI 设备。可以用 i2c-tools 来获取 twi 设备的相关信息（默认集成在内核里面），并且读写相关的 twi 设备的数据。

i2c-tools 主要是通过读写 /dev/i2c-* 文件获取 TWI 设备，所以需要在 menuconfig 里面把 TWI 的 device interface 节点打开，具体的 i2c-tools 使用方法如下：

6.1.1 i2cdetect

6.1.1.1 参数说明

```
# ./i2cdetect
Usage: i2cdetect [-y] [-a] [-q|-r] TWIBUS [FIRST LAST]
       i2cdetect -F TWIBUS
       i2cdetect -l
TWIBUS is an integer or an TWI bus name
If provided, FIRST and LAST limit the probing range.
```

- -y: 关闭交互式，不会显示警告信息
- -a: 扫描总线上所有设备
- -q: 使用 SMBus 的 “quick write” 命令进行检测，不建议使用
- -r: 使用 SMBus 的 “receive byte” 命令进行检测，不建议使用
- TWIBUS: 指定查询某个总线编号
- FIRST LAST: 扫描的地址范围

6.1.1.2 使用示例

在小机端运行:

```
./i2cdetect -l
```

运行结果如下图所示，探测设备中所有的 i2c 总线：

6.1.2 i2cdump

6.1.2.1 参数说明

```
# ./i2cdump
Usage: i2cdump [-f] [-y] [-r first-last] [-a] TWIBUS ADDRESS [MODE [BANK [BANKREG]]]
TWIBUS is an integer or an TWI bus name
ADDRESS is an integer (0x08 - 0x77, or 0x00 - 0x7f if -a is given)
MODE is one of:
b (byte, default)
w (word)
W (word on even register addresses)
s (SMBus block, deprecated)
i (TWI block)
c (consecutive byte)
Append p for SMBus PEC
```

- -r: 指定寄存器范围，只能扫描从 first 到 last 区域;
- -f: 强制访问设备;
- -y: 关闭人机交互模式
- TWIBUS: 总线编号
- ADDRESS: 指定设备地址
- MODE: 指定读取的大小，b 字节，w 字，s 是 SMBus 块，i 是 twi 块

6.1.2.2 使用示例

在小机端运行:

```
# ./i2cdump -f -y 0 0x50
```

运行结果如下图所示，打印出 i2c-0 总线上 0x50 设备中所有寄存器的值:

6.1.3 i2cset

6.1.3.1 参数说明

```
# ./i2cset
Usage: i2cset [-f] [-y] [-m MASK] [-r] [-a] TWIBUS CHIP-ADDRESS DATA-ADDRESS [VALUE] ... [MODE]
TWIBUS is an integer or an TWI bus name
ADDRESS is an integer (0x08 - 0x77, or 0x00 - 0x7f if -a is given)
MODE is one of:
c (byte, no value)
b (byte data, default)
w (word data)
```

i (TWI block data)
s (SMBus block data)
Append p for SMBus PEC

- -r: 写入后立即回读寄存器的值，并将结果与写入的值进行比较;
- TWIBUS: 总线编号;
- CHIP-ADDRESS:twi 设备地址;
- DATA-ADDRESS:twi 寄存器地址;
- VALUE: 要写入的值
- MODE: 指定读取的大小，b 字节，w 字，s 是 SMBus 块，i 是 twi 块

6.1.3.2 使用示例

在小机端运行:

```
# ./i2cset -f -y 0 0x50 0x00 0xff
```

运行结果如下图所示，向 i2c-0 总线上 0x50 设备中地址为 0x00 寄存器中写入 0xff 数据:

6.1.4 i2cget

6.1.4.1 参数说明

```
# ./i2cget
Usage: i2cget [-f] [-y] [-a] TWIBUS CHIP-ADDRESS [DATA-ADDRESS [MODE [LENGTH]]]
TWIBUS is an integer or an TWI bus name
ADDRESS is an integer (0x08 - 0x77, or 0x00 - 0x7f if -a is given)
MODE is one of:
    b (read byte data, default)
    w (read word data)
    c (write byte/read byte)
    s (read SMBus block data)
    i (read TWI block data)
Append p for SMBus PEC
LENGTH is the TWI block data length (between 1 and 32, default 32)
```

- -f: 强制访问
- -y: 关闭交互模式，不会提示警告信息
- TWIBUS: 总线编号
- CHIP-ADDRESS:twi 设备地址
- DATA-ADDRESS:twi 寄存器地址
- MODE: 指定读取的大小，b 字节，w 字，s 是 SMBus 块，i 是 twi 块

6.1.4.2 使用示例

在小机端运行:

```
# ./i2cget -f -y 0 0x50 0x00
```

运行结果如下图所示，读取 i2c-0 总线上 0x50 设备中地址为 0x00 寄存器的值:

6.1.5 i2ctransfer

注意：i2ctransfer 接口是在 v4.0 版本之后才支持的，因要使用该命令需要获取 4.0 以上的 i2c-tools 工具；源码包可在下方网站中进行获取（需注意，要获取 4.0 及以上版本）：

```
https://mirrors.edge.kernel.org/pub/software/utis/i2c-tools/
```

6.1.5.1 参数说明

```
# ./i2ctransfer
Usage: i2ctransfer [-f] [-y] [-v] [-V] [-a] TWIBUS DESC [DATA] [DESC [DATA]]...
TWIBUS is an integer or an TWI bus name
DESC describes the transfer in the form: {r|w}LENGTH[@address]
  1) read/write-flag 2) LENGTH (range 0-65535, or '?')
  3) TWI address (use last one if omitted)
DATA are LENGTH bytes for a write message. They can be shortened by a suffix:
= (keep value constant until LENGTH)
+ (increase value by 1 until LENGTH)
- (decrease value by 1 until LENGTH)
p (use pseudo random generator until LENGTH with value as seed)

Example (bus 0, read 8 byte at offset 0x64 from EEPROM at 0x50):
# i2ctransfer 0 w1@0x50 0x64 r8
Example (same EEPROM, at offset 0x42 write 0xff 0xfe ... 0xf0):
# i2ctransfer 0 w17@0x50 0x42 0xff
```

6.1.5.2 使用示例

在小机端运行

```
# ./i2ctransfer -f -y 0 w42@0x50 0x00 0xaa=
```

运行结果如下图所示，向 i2c-0 总线的 0x50 设备的 0x00 地址写入 41 个字节的数据，数据内容为 0xaa(因为是 drv-mode，且传输数据数量大于 32 个字节，因此采用 DMA 传输)：

6.2 调试节点

6.2.1 info

此节点文件可以打印出当前 TWI 通道的一些硬件资源信息。

```
cat /sys/devices/platform/soc*/twi*/info
```

6.2.2 status

此节点文件可以打印出当前 TWI 通道的一些运行状态信息，包括控制器的各寄存器值。

```
cat /sys/devices/platform/soc*/twi*/status
```

6.2.3 freq

此节点文件可以动态修改 twi 频率，目前只支持输入 **100000/200000/400000**。

```
echo 100000 > /sys/devices/platform/soc*/twi*/freq
```

6.2.4 twi_mode

此节点文件可以查看并动态修改 twi 的工作模式。

```
$ cat twi_mode # 查看此时的工作状态
$ Current mode is 1: drv # 显示此时为drv模式
$ echo 0 > twi_mode # 将工作模式修改为engine模式（向节点写0）
$ sunxi-twi 2502000.twi0: Change twi mode from drv to engine
$ echo 1 > twi_mode # 将工作模式修改为drv模式（向节点写1）
$ sunxi-twi 2502000.twi0: Change twi mode from engine to drv
```

6.2.5 debug_chan

此节点文件可以指定某 twi 输出 debug 信息：

```
$ cat debug_chan # 查看此时支持哪几路twi打印debug信息
$ Current debug_chan_flags = 0x0 # 默认不打开所以twi的debug打印

# | twi | ... | twi7 | twi6 | twi5 | twi4 | twi3 | twi2 | twi1 | twi0 |
# | bits | ... | bit7 | bit6 | bit5 | bit4 | bit3 | bit2 | bit1 | bit0 |
# 如果需要twi4和twi5打印debug信息，则让debug_chan_flags的bit4和bit5都为1，因此debug_chan_flags=0x30，也就是说，如果你希望让twi*打印debug信息，你就需要让debug_chan_flags的对应的bit位为1，然后将计算得到的16进制数，echo到debug_chan节点
$ echo 0x30 > debug_chan
```

7 FAQ

7.1 TWI 数据未完全发送

问题现象：incomplete xfer。具体的 log 如下所示：

```
[ 1658.926643] sunxi_i2c_do_xfer()1936 - [i2c0] incomplete xfer (status: 0x20, dev addr: 0x50)
[ 1658.926643] sunxi_i2c_do_xfer()1936 - [i2c0] incomplete xfer (status: 0x48, dev addr: 0x50)
```

问题分析：此错误表示主控已经发送了数据（status 值为 0x20 时，表示发送了 SLAVE ADDR + WRITE；status 值为 0x48 时，表示发送了 SLAVE ADDR + READ），但是设备没有回 ACK，这表明设备无响应，应该检查是否未接设备、接触不良、设备损坏和上电时序不正确导致的设备未就绪等问题。

问题排查步骤：

- 步骤 1：通过设备树里面的配置信息，核对引脚配置是否正确。每组 TWI 都有好几组引脚配置。
- 步骤 2：更换 TWI 总线下的设备为 at24c16，用 i2ctools 读写 at24c16 看看是否成功，成功则表明总线工作正常。
- 步骤 3：排查设备是否可以正常工作以及设备与 I2C 之间的硬件接口是否完好。
- 步骤 4：详细了解当前需要操作的设备的初始化方法，工作时序，使用方法，排查因初始化设备不正确导致通讯失败。
- 步骤 5：用示波器检查 TWI 引脚输出波形，查看波形是否匹配。

7.2 TWI 起始信号无法发送

问题现象：START can't sendout!。具体的 log 如下所示：

```
sunxi_i2c_do_xfer()1865 - [i2c1] START can't sendout!
```

问题分析：此错误表示 TWI 无法发送起始信号，一般跟 TWI 总线的引脚配置以及时钟配置有关。应该检查引脚配置是否正确，时钟配置是否正确，引脚是否存在上拉电阻等等。

问题排查步骤：

- 步骤 1：重新启动内核，通过查看 log，分析 TWI 是否成功初始化，如若存在引脚配置问题，应核对引脚信息是否正确。

- 步骤 2：根据原理图，查看 TWI-SCK 和 TWI-SDA 是否经过合适的上拉电阻接到 3.3v 电压。
- 步骤 3：用万用表量 SDA 与 SCL 初始电压，看电压是否在 3.3V 附近（断开此 TWI 控制器所有外设硬件连接与软件通讯进程）。
- 步骤 4：核查引脚配置以及 clk 配置是否进行正确设置。
- 步骤 5：测试 PIN 的功能是否正常，利用寄存器读写的方式，将 PIN 功能直接设为 INPUT 功能（echo [reg] [val] > /sys/class/sunxi_dump/write），然后将 PIN 上拉和接地改变 PIN 状态，读 PIN 的状态（echo [reg,reg] > /sys/class/sunxi_dump/dump;cat dump），看是否匹配。
- 步骤 6：测试 CLK 的功能是否正常，利用寄存器读写的方式，将 TWI 的 CLK gating 等打开，（echo [reg] [val] > /sys/class/sunxi_dump/write），然后读取相应 TWI 的寄存器信息，读 TWI 寄存器的数据（echo [reg] , [len] > /sys/class/sunxi_dump/dump），查看寄存器数据是否正常。

7.3 TWI 终止信号无法发送

问题现象：STOP can't sendout。具体的 log 如下所示：

```
twi_stop()511 - [i2c4] STOP can't sendout!  
sunxi_i2c_core_process()1726 - [i2c4] STOP failed!
```

问题分析：此错误表示 TWI 无法发送终止信号，一般跟 TWI 总线的引脚配置。应该检查引脚配置是否正确，引脚电压是否稳定等等。

问题排查步骤：

- 步骤 1：根据原理图，查看 TWI-SCK 和 TWI-SDA 是否经过合适的上拉电阻接到 3.3v 电压。
- 步骤 2：用万用表量 SDA 与 SCL 初始电压，看电压是否在 3.3V 附近（断开此 TWI 控制器所有外设硬件连接与软件通讯进程）。
- 步骤 3：测试 PIN 的功能是否正常，利用寄存器读写的方式，将 PIN 功能直接设为 INPUT 功能（echo [reg] [val] > /sys/class/sunxi_dump/write），然后将 PIN 上拉和接地改变 PIN 状态，读 PIN 的状态（echo [reg,reg] > /sys/class/sunxi_dump/dump;cat dump），看是否匹配。
- 步骤 4：查看设备树配置，把其他用到 SCK/SDA 引脚的节点关闭，重新测试 I2C 通信功能。

7.4 TWI 传送超时

问题现象：xfer timeout。具体的 log 如下所示：

```
[123.681219] sunxi_i2c_do_xfer()1914 - [i2c3] xfer timeout (dev addr:0x50)
```

问题分析：此错误表示主控已经发送完起始信号，但是在与设备通信的过程中无法正常完成数据发送与接收，导致最终没有发出终止信号来结束 I2C 传输，导致的传输超时问题。应该检查引脚配置

是否正常，CLK 配置是否正常，TWI 寄存器数据是否正常，是否有其他设备干扰，中断是否正常等问题。

问题排查步骤：

- 步骤 1：核实 TWI 控制器配置是否正确。
- 步骤 2：根据原理图，查看 TWI-SCK 和 TWI-SDA 是否经过合适的上拉电阻接到 3.3v 电压。
- 步骤 3：用万用表量 SDA 与 SCL 初始电压，看电压是否在 3.3V 附近（断开此 TWI 控制器所有外设硬件连接与软件通讯进程）。
- 步骤 4：关闭其他 TWI 设备，重新进行烧录测试 TWI 功能是否正常。
- 步骤 5：测试 PIN 的功能是否正常，利用寄存器读写的方式，将 PIN 功能直接设为 INPUT 功能（`echo [reg] [val] > /sys/class/sunxi_dump/write`），然后将 PIN 上拉和接地改变 PIN 状态，读 PIN 的状态（`echo [reg,reg] > /sys/class/sunxi_dump/dump;cat dump`），看是否匹配。
- 步骤 6：测试 CLK 的功能是否正常，利用寄存器读写的方式，将 TWI 的 CLK gating 等打开，（`echo [reg] [val] > /sys/class/sunxi_dump/write`），然后读取相应 TWI 的寄存器信息，读 TWI 寄存器的数据（`echo [reg] ,[len]> /sys/class/sunxi_dump/dump`），查看寄存器数据是否正常。
- 步骤 7：根据相关的 LOG 跟踪 TWI 代码执行流程，分析报错原因。

著作权声明

版权所有 © 2023 珠海全志科技股份有限公司。保留一切权利。

本文档及内容受著作权法保护，其著作权由珠海全志科技股份有限公司（“全志”）拥有并保留一切权利。

本文档是全志的原创作品和版权财产，未经全志书面许可，任何单位和个人不得擅自摘抄、复制、修改、发表或传播本文档内容的部分或全部，且不得以任何形式传播。

商标声明



（不完全列举）均为珠海全志科技股份有限公司的商标或者注册商标。在本文档描述的产品中出现的其它商标，产品名称，和服务名称，均由其各自所有人拥有。

免责声明

您购买的产品、服务或特性应受您与珠海全志科技股份有限公司（“全志”）之间签署的商业合同和条款的约束。本文档中描述的全部或部分产品、服务或特性可能不在您所购买或使用的范围内。使用前请认真阅读合同条款和相关说明，并严格遵循本文档的使用说明。您将自行承担任何不当使用行为（包括但不限于如超压，超频，超温使用）造成的不利后果，全志概不负责。

本文档作为使用指导仅供参考。由于产品版本升级或其他原因，本文档内容有可能修改，如有变更，恕不另行通知。全志尽全力在本文档中提供准确的信息，但并不确保内容完全没有错误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或发生侵犯第三方权利事件，全志概不负责。本文档中的所有陈述、信息和建议并不构成任何明示或暗示的保证或承诺。

本文档未以明示或暗示或其他方式授予全志的任何专利或知识产权。在您实施方案或使用产品的过程中，可能需要获得第三方的权利许可。请您自行向第三方权利人获取相关的许可。全志不承担也不代为支付任何关于获取第三方许可的许可费或版税（专利税）。全志不对您所使用的第三方许可技术做出任何保证、赔偿或承担其他义务。